MULTI-AGENT ARCHITECTURE

From Beginner to Master

JIALU CHENG

2026-04-21

Multi-Agent Architecture

作者： Jialu Cheng

出版： X: Jialu Cheng (@Xialu8888)

版本： v1.0

日期： 2026-04-21

简介： The complete guide to multi-agent systems. 10 chapters covering LangGraph, CrewAI, OpenAI SDK, AutoGen, Google ADK, MCP & A2A protocols. From single-agent limitations to production deployment.

版权所有 © 2026 Jialu Cheng。保留所有权利。
未经书面许可，不得以任何形式复制或传播本书内容。

目录

Multi-Agent Architecture

From Beginner to Master

Part 1: Getting Started

§ 01 Single Agent Hits the Wall

§ 02 Architecture Patterns: Pick Wrong, Start Over

§ 03 First Project: Research Assistant with LangGraph

Part 2: Core Concepts

§ 04 Communication: Message Passing, State & Handoff

§ 05 Framework Comparison

§ 06 MCP & A2A: The Protocol Layer

Part 3: Production

§ 07 Production: Multi-Agent Support System

§ 08 Debugging & Observability

§ 09 Cost Control

§ 10 The Future: From Tools to Coworkers

Appendix

MULTI-AGENT ARCHITECTURE

From Beginner to Master

Author: Jialu Cheng
X: @Xialu8888
For: Developers, architects, and product managers moving from single-agent to multi-agent systems
Stack: LangGraph / CrewAI / OpenAI Agents SDK / AutoGen / Google ADK / MCP / A2A (2025-2026)
Updated: 2026-04-21
Use Cases: Understanding multi-agent patterns, framework selection, building your first system, avoiding production pitfalls

---

Part 1: Getting Started

Zero to one. Three chapters to understand what multi-agent actually means and ship your first project.

---

§ 01 Single Agent Hits the Wall

01.1 My Single-Agent Breaking Point

March 2025. I was building a full-stack project with Claude.

Single agent worked fine for small tasks. Write an API endpoint, tweak some frontend, run tests. It handled everything. But when the project grew, the cracks showed.

I asked it to simultaneously update backend APIs, frontend calls, and integration tests. It started losing context. Fixed file A, forgot file B. Wrote the interface, skipped the tests.

This isn’t a model limitation. It’s a structural bottleneck of single-agent architecture.

An agent’s context window is finite. 128k tokens sounds massive, but a mid-sized codebase easily exceeds 500k tokens. Asking one agent to simultaneously understand database schemas, API contracts, frontend components, and test suites—it doesn’t work.

> ***My Take**: The Single-Agent Ceiling*
>
> *I developed with a single agent for 47 days straight. For 12 of those days, I had to repeatedly feed context back: “You forgot, this file was already modified.” Each recovery conversation wasted 15-30 minutes. Over 47 days, I lost approximately 14 hours to context remediation.*

01.2 What Multi-Agent Actually Solves

TL;DR: Multi-agent isn’t about spending more on API calls. It’s about giving each agent a clean, focused context to do what it does best.

Single agent is the solo full-stack engineer. Jack of all trades, master of none.

Multi-agent is a specialized team. One handles database optimization, one owns API design, one crafts frontend components, one maintains test coverage. Each masters their domain.

ASPECT	SINGLE AGENT	MULTI-AGENT	VERDICT
Context Management	Everything crammed into one window	Each agent has isolated context	Multi-agent preserves context
Specialization	Surface-level knowledge of everything	Deep prompt customization per agent	Higher quality output
Parallelism	Sequential execution	Multiple agents run in parallel	3-5x faster
Failure Isolation	One mistake corrupts everything	Errors contained per agent	Better reliability

01.3 Three Signals You Need Multi-Agent

Not every project needs multi-agent. Here’s when you do:

Signal 1: Context Pollution

Your agent keeps forgetting what you told it 10 minutes ago. You’re repeatedly saying “remember that file we modified earlier?” If this happens more than twice per session, you need separation.

Signal 2: Parallel Workflows

You have tasks that don’t depend on each other. Research and writing can happen simultaneously. A single agent does them sequentially. Multiple agents do them in parallel.

Signal 3: Specialized Domains

Your project spans fundamentally different domains. A coding agent and a research agent need different tool sets, different prompts, different evaluation criteria. Forcing them into one context dilutes both.

> ***Key Advice**: Start with single agent. Scale to multi-agent only when you hit these signals. Premature optimization wastes more time than it saves.*

01.4 The Cost Reality

Multi-agent costs more. But not as much as you think.

SCENARIO	SINGLE AGENT	MULTI-AGENT	OVERHEAD
Simple CRUD API	1x	1.2x	20%
Full-stack feature	1x	2.5x	150%
Research + Write + Review	1x (sequential)	3x (parallel)	200% but 3x faster

The overhead comes from:

- Inter-agent communication (passing context between agents)
- Coordination logic (orchestrator, state management)
- Redundant context (each agent needs shared baseline context)

But the tradeoff is worth it when:

- Quality matters more than cost
- Time-to-completion is critical
- Context pollution is killing your productivity

01.5 Forward: Architecture Patterns

Understanding when to use multi-agent is step one. Step two is picking the right architecture pattern. Orchestrator-Worker, Pipeline, Hierarchical, Swarm—each solves different problems.

Next chapter: how to split tasks correctly. Get this wrong and your multi-agent system becomes a distributed mess.

§ 02 Architecture Patterns: Pick Wrong, Start Over

02.1 The Five Patterns

Multi-agent architecture isn’t one-size-fits-all. Here are the five patterns that cover 95% of use cases:

PATTERN	STRUCTURE	CONTROL	BEST FOR
Orchestrator-Worker	One boss, many workers	Centralized	Task decomposition with clear sub-tasks
Pipeline	Linear chain: A → B → C	Sequential	Document processing, ETL workflows
Hierarchical	Tree structure	Top-down	Complex decision-making, approval flows
Swarm	Peer-to-peer mesh	Distributed	Exploration tasks, consensus building
Router	Dynamic routing based on input	Conditional	Multi-domain systems, customer support

02.2 Orchestrator-Worker (Most Common)

One agent decomposes the task. Workers execute in parallel. Orchestrator synthesizes results.

```python
# orchestrator.py
class Orchestrator:
    def __init__(self):
        self.researcher = ResearcherAgent()
        self.writer = WriterAgent()
        self.reviewer = ReviewerAgent()

    def execute(self, task):
        # Decompose
        subtasks = self.decompose(task)

        # Parallel execution
        results = await asyncio.gather(
            self.researcher.run(subtasks[0]),
            self.writer.run(subtasks[1]),
            self.reviewer.run(subtasks[2])
        )

        # Synthesize
        return self.synthesize(results)
```

When to use: Clear task boundaries, parallelizable subtasks, need for centralized coordination.

When to avoid: Subtasks are highly interdependent, real-time collaboration required.

02.3 Pipeline (Data Processing)

Linear transformation. Output of stage N is input to stage N+1.

```python
# pipeline.py
pipeline = [
    ExtractAgent(),      # Pull data from source
    TransformAgent(),    # Clean and structure
    ValidateAgent(),     # Check quality
    LoadAgent()          # Save to destination
]

result = input_data
for agent in pipeline:
    result = await agent.process(result)
```

When to use: Document processing, data ETL, content moderation, sequential transformations.

When to avoid: Need for parallel processing, conditional branching, feedback loops.

02.4 Context-Centric Task Decomposition

This is the critical insight from Anthropic’s research (Jan 2026): **split by context, not by function**.

Bad split (problem-centric):

- Writer Agent + Editor Agent
- Why bad? Both need the full document context
- Result: Context duplication, overhead without benefit

Good split (context-centric):

- Research Agent (needs search context)
- Writer Agent (needs writing context)
- Verifier Agent (needs validation context only)

SCENARIO	BAD SPLIT	GOOD SPLIT	WHY
Content creation	Writer + Editor	Researcher + Writer	Editor needs same context as writer
Code review	Coder + Reviewer	Coder + Tester + Security	Each has distinct context needs
Data analysis	Extractor + Analyzer	Extractor + Validator + Analyzer	Validation is black-box

02.5 The Verification Subagent Pattern

A special case of context-centric splitting. Verification often needs different context than creation.

The “Early Victory” Problem: If a verification agent sees how the answer was constructed, it may approve prematurely. Solution: give it only the output, not the process.

```python
# verification_pattern.py
class VerifiedWorkflow:
    def __init__(self):
        self.researcher = ResearcherAgent()
        self.writer = WriterAgent()
        # Verifier gets isolated context
        self.verifier = VerifierAgent(seen_context=False)

    async def run(self, topic):
        research = await self.researcher.gather(topic)
        draft = await self.writer.compose(research)
        # Verifier only sees draft, not research process
        valid = await self.verifier.check(draft)
        return draft if valid else await self.rewrite(draft, valid.feedback)
```

02.6 Forward: Building Your First System

You know the patterns. Now let’s build something real. Next chapter: a complete research assistant using LangGraph, implementing the context-centric principles we just covered.

---

§ 03 First Project: Research Assistant with LangGraph

03.1 Project Overview

We’re building a research assistant with three agents:

1. **Researcher**: Gathers information (isolated search context)
2. **Writer**: Composes the report (writing context, shares some with Researcher)
3. **Verifier**: Validates facts (black-box verification, minimal context)

Notice: No separate Editor agent. Writer does self-editing because editing requires the same context as writing.

03.2 State Design

```python
# state.py
from typing import TypedDict, Annotated
import operator

class ResearchState(TypedDict):
    topic: str
    search_results: Annotated[list, operator.add]
    draft: str
    validation_report: dict
    final_output: str
    iterations: int
```

03.3 Agent Definitions

```python
# agents.py
from langchain_openai import ChatOpenAI
from langgraph.graph import StateGraph

class ResearcherAgent:
    def __init__(self):
        self.llm = ChatOpenAI(model="gpt-4o")

    async def run(self, state: ResearchState):
        """Gather information. Only needs search context."""
        query = f"Research: {state['topic']}"
        results = await search_web(query)
        return {"search_results": results}

class WriterAgent:
    def __init__(self):
        self.llm = ChatOpenAI(model="gpt-4o")

    async def run(self, state: ResearchState):
        """Compose report. Shares context with Researcher (results)."""
        prompt = f"Write report on {state['topic']} using: {state['search_results']}"
        draft = await self.llm.ainvoke(prompt)
        return {"draft": draft.content}

class VerifierAgent:
    def __init__(self):
        self.llm = ChatOpenAI(model="gpt-4o-mini")  # Cheaper for verification

    async def run(self, state: ResearchState):
        """Validate facts. Only sees draft, not research process."""
        prompt = f"Verify factual accuracy: {state['draft']}"
        validation = await self.llm.ainvoke(prompt)
        return {"validation_report": parse_validation(validation)}
```

03.4 Graph Construction

```python
# graph.py
from langgraph.graph import StateGraph, END

builder = StateGraph(ResearchState)

# Add nodes
builder.add_node("research", ResearcherAgent().run)
builder.add_node("write", WriterAgent().run)
builder.add_node("verify", VerifierAgent().run)

# Define flow
builder.set_entry_point("research")
builder.add_edge("research", "write")
builder.add_edge("write", "verify")

# Conditional: retry if verification fails
builder.add_conditional_edges(
    "verify",
    lambda state: "accept" if state["validation_report"]["pass"] else "retry",
    {"accept": END, "retry": "write"}
)

graph = builder.compile()
```

03.5 Running It

```python
# main.py
import asyncio

async def main():
    result = await graph.ainvoke({
        "topic": "Multi-agent architecture patterns 2025",
        "iterations": 0
    })
    print(result["final_output"])

if __name__ == "__main__":
    asyncio.run(main())
```

03.6 Key Takeaways

1. **Context isolation**: Each agent only gets what it needs

2. **No Editor agent**: Writer self-edits (shared context)
3. **Verifier isolation**: Black-box validation prevents "early victory"
4. **Conditional loops**: Verification failures trigger rewrites

> ***My Take***: *This three-agent setup replaced my single-agent research workflow. Time per report dropped from 45 minutes to 12 minutes. Quality improved because the Verifier catches facts I'd miss.*

Part 2: Core Concepts

Three chapters on communication mechanisms, framework selection, and the protocol layer.

---

§ 04 Communication: Message Passing, State & Handoff

04.1 Three Communication Models

Agents need to talk. Here are the three patterns:

MODEL	MECHANISM	LATENCY	COMPLEXITY	USE CASE
Message Passing	Direct send/receive	Low	Low	One-to-one coordination
Shared State	Centralized store	Medium	Medium	Multi-agent synchronization
Handoff	Control transfer	Low	High	Stateful conversations

04.2 Message Passing

Direct async messages between agents.

```
# Simple message passing
async def agent_a(message: str, recipient: Agent):
    processed = await self.llm.ainvoke(f"Process: {message}")
    await recipient.receive(processed.content)
```

Pros: Simple, explicit, easy to debug

Cons: Tight coupling, doesn't scale to many agents

04.3 Shared State (LangGraph Style)

Centralized state object that all agents read/write.

```
# LangGraph shared state
class WorkflowState(TypedDict):
    messages: Annotated[list, add_messages]
    current_agent: str
    final_answer: str

# Agents read/write state
class Agent:
    def run(self, state: WorkflowState):
        # Read from state
        history = state["messages"]
        # Process
        response = self.llm.invoke(history)
        # Write to state
        return {"messages": [response], "current_agent": self.name}
```

Pros: Loose coupling, audit trail, easy replay

Cons: State management complexity, potential conflicts

04.4 Handoff (OpenAI Swarm Style)

Agent transfers control to another agent.

```python
# Swarm-style handoff
from swarm import Swarm, Agent

def transfer_to_researcher():
    """Handoff function called when research needed."""
    return researcher_agent

orchestrator = Agent(
    name="Orchestrator",
    instructions="Route tasks to specialists",
    functions=[transfer_to_researcher, transfer_to_writer]
)

client = Swarm()
result = client.run(
    agent=orchestrator,
    messages=[{"role": "user", "content": "Research LLM trends"}]
)
# Automatically hands off to researcher_agent
```

Pros: Natural conversation flow, clear ownership

Cons: Complex debugging, potential infinite loops

---

§ 05 Framework Comparison

05.1 The Contenders

FRAMEWORK	ARCHITECTURE	LEARNING CURVE	PRODUCTION READY	BEST FOR
LangGraph	Graph-based	Medium	Yes	Complex workflows, state management
CrewAI	Role-based	Low	Yes	Quick prototypes, roleplay scenarios
OpenAI SDK	Swarm/Handoff	Low	Yes	Simple handoffs, OpenAI ecosystem
AutoGen/AG2	Conversational	High	Maybe	Research, multi-turn conversations
Google ADK	Service-based	Medium	Early	GCP deployments, enterprise

05.2 LangGraph Deep Dive

Strengths:

- Explicit graph visualization
- Built-in persistence
- Conditional edges
- Human-in-the-loop support

Weaknesses:

- Verbose setup

- Steep learning curve
- Overkill for simple workflows

```
# LangGraph conditional routing
builder.add_conditional_edges(
    "analyze",
    route_based_on_confidence,
    {
        "high": "respond",
        "medium": "clarify",
        "low": "research"
    }
)
```

05.3 CrewAI Deep Dive

Strengths:

- Role-based configuration
- Simple YAML setup
- Built-in task delegation

Weaknesses:

- Less flexible than LangGraph
- Harder to customize execution flow

```
# CrewAI setup
from crewai import Agent, Crew, Task

researcher = Agent(role="Researcher", goal="Gather data")
writer = Agent(role="Writer", goal="Create content")

task = Task(description="Research and write", agent=researcher)
crew = Crew(agents=[researcher, writer], tasks=[task])
result = crew.kickoff()
```

05.4 Decision Framework

Choose based on your constraints:

```
Need complex state machine? → LangGraph
Want quick role-based setup? → CrewAI
Using OpenAI exclusively? → OpenAI SDK
Research/experimental? → AutoGen
GCP enterprise deploy? → Google ADK
```

---

§ 06 MCP & A2A: The Protocol Layer

06.1 MCP (Model Context Protocol)

Anthropic’s open standard for connecting AI systems to data sources.

Core idea: Tools expose themselves via MCP servers. Agents connect as clients.

```python
# MCP Server (tool provider)
from mcp.server import Server

server = Server("database-mcp")

@server.tool()
async def query(sql: str) -> str:
    return db.execute(sql)

# MCP Client (agent)
from mcp.client import Client

client = Client()
await client.connect("database-mcp")
result = await client.call("query", {"sql": "SELECT * FROM users"})
```

Why it matters: Decouples tools from agents. One MCP server, many agents.

06.2 A2A (Agent-to-Agent Protocol)

Google’s protocol for agent interoperability.

Core idea: Standardized way for agents to discover and communicate.

```
# A2A Agent Card (self-description)
{
    "name": "ResearchAgent",
    "capabilities": ["web_search", "summarize"],
    "endpoint": "https://agent.example.com/a2a",
    "authentication": {"type": "api_key"}
}

# A2A Task submission
task = {
    "id": "task-123",
    "message": {"role": "user", "parts": [{"text": "Research AI trends"}]},
    "artifacts": []
}
```

Why it matters: Agents from different vendors can collaborate.

06.3 Protocol Stack

LAYER	PROTOCOL	PURPOSE
Application	A2A	Agent discovery & collaboration
Service	MCP	Tool integration
Transport	HTTP/SSE	Communication

Part 3: Production

Four chapters on building real systems, debugging, cost control, and future trends.

---

§ 07 Production: Multi-Agent Support System

07.1 System Architecture

Customer support system with three agents:

1. **Router**: Classifies incoming tickets
2. **Billing Agent**: Handles payment issues
3. **Technical Agent**: Handles product bugs

```
# router_agent.py
class RouterAgent:
    def route(self, ticket: Ticket) -> Agent:
        category = self.classify(ticket.content)
        return {
            "billing": self.billing_agent,
            "technical": self.tech_agent
        }[category]
```

07.2 Confidence-Based Routing

Don’t blindly route. Use confidence scores.

```
# confidence.py
class SmartRouter:
    def route_with_confidence(self, ticket):
        categories = self.classify(ticket)

        if categories[0].confidence > 0.8:
            return categories[0].agent
        elif categories[0].confidence > 0.5:
            # Human review for uncertain cases
            return self.human_agent
        else:
            # Default to generalist
            return self.general_agent
```

§ 08 Debugging & Observability

08.1 The Debugging Challenge

Multi-agent bugs are harder than single-agent bugs:

- Which agent made the mistake?
- Was it the input or the agent logic?
- How did state evolve across the graph?

08.2 Essential Observability

```python
# observable_pipeline.py
from langsmith import traceable

@traceable
def agent_node(state):
    # Automatically logged to LangSmith
    return agent.run(state)

# Key metrics to track
METRICS = {
    "latency_per_agent": {},
    "token_usage_per_agent": {},
    "error_rate": 0,
    "retry_count": 0
}
```

08.3 The 14 Failure Modes (NeurIPS 2025)

Research shows multi-agent failures cluster in three categories:

CATEGORY	% OF FAILURES	EXAMPLES
System Design	40%	Wrong architecture, poor task split
Inter-agent Misalignment	35%	Communication errors, goal conflict
Task Verification	25%	Wrong verification, missed edge cases

> ***My Take***: *8 hours debugging one punctuation error in an agent message. Now I log everything. Observability isn't optional—it's the cost of admission.*

---

§ 09 Cost Control

09.1 Cost Breakdown

COMPONENT	TYPICAL %	OPTIMIZATION
Model calls	70%	Model tiering, caching
Retries	15%	Better error handling
Orchestration	10%	Efficient state mgmt
Overhead	5%	Minimize inter-agent chatter

09.2 Model Tiering

Use the right model for the right task.

```
# Model tiering strategy
TIERING = {
    "complex_reasoning": "gpt-4o",       # $5/1M tokens
    "standard_tasks": "gpt-4o-mini",     # $0.15/1M tokens
    "simple_validation": "gpt-4o-mini",  # $0.15/1M tokens
}

# Savings: ~70% on validation tasks
```

09.3 Semantic Caching

Cache similar queries.

```python
# Simple semantic cache
from sentence_transformers import SentenceTransformer

class SemanticCache:
    def __init__(self):
        self.encoder = SentenceTransformer('all-MiniLM-L6-v2')
        self.cache = {}

    def get(self, query: str) -> str | None:
        embedding = self.encoder.encode(query)
        # Find similar cached queries
        for cached_emb, result in self.cache.items():
            if cosine_similarity(embedding, cached_emb) > 0.95:
                return result
        return None
```

---

§ 10 The Future: From Tools to Coworkers

10.1 Current State

We’re at the “smart tools” phase:

- Agents execute tasks
- Humans specify goals
- One-way interaction

10.2 Emerging Patterns

Autonomous Teams: Agents self-organize around goals.

```
Current: Human assigns → Agents execute → Human reviews
Future:  Agents propose → Human approves → Agents execute
```

Persistent Agents: Long-running agents with memory across sessions.

Cross-Organization Agents: Agents from different companies collaborating via A2A.

10.3 Final Thoughts

Multi-agent isn’t hype. It’s the natural evolution from single-agent limitations.

Start simple. One orchestrator, two workers. Master context-centric splitting. Add complexity only when needed.

The future isn’t one super-agent. It’s a team of specialized agents, each excellent at one thing, collaborating seamlessly.

> *By Jialu Cheng | AI Native Coder · Independent Developer*
> *X: @Xialu8888*
> *Projects: Claude Code Mastery · OpenClaw Orange Book · Hermes Agent Guide*

Appendix

Framework Cheat Sheet

TASK	RECOMMENDED	WHY
Complex workflow	LangGraph	State management, visualization
Quick prototype	CrewAI	Role-based, fast setup
OpenAI stack	OpenAI SDK	Native integration
Research	AutoGen	Multi-turn conversations
Enterprise GCP	Google ADK	Managed deployment

Decision Tree

```
Is your workflow linear?
├── Yes → Pipeline pattern
└── No
    └── Need centralized control?
        ├── Yes → Orchestrator-Worker
        └── No
            └── Agents need to negotiate?
                ├── Yes → Swarm
                └── No → Router
```

Protocol Comparison

PROTOCOL	VENDOR	STATUS	USE CASE
MCP	Anthropic	Stable	Tool integration
A2A	Google	Emerging	Agent collaboration
ACP	Community	Draft	Standardization

Cost Template

```python
# Quick cost estimator
def estimate_cost(agents, queries_per_day, avg_tokens):
    """
    agents: dict of {agent_name: model_name}
    queries_per_day: int
    avg_tokens: average tokens per query
    """
    PRICING = {
        "gpt-4o": 5.0,        # $ per 1M tokens
        "gpt-4o-mini": 0.15,
        "claude-3-opus": 15.0,
    }

    daily_cost = 0
    for agent, model in agents.items():
        cost_per_query = (avg_tokens / 1_000_000) * PRICING[model]
        daily_cost += cost_per_query * queries_per_day

    return daily_cost * 30  # Monthly estimate
```

Reading Guide

Day 1 (Chapters 1-3): Understand the problem, learn patterns, build first project

Day 2 (Chapters 4-6): Communication models, framework selection, protocols

Day 3 (Chapters 7-10): Production systems, debugging, cost optimization, future trends

---

Last updated: 2026-04-21

Version: 1.0

MULTI-AGENT ARCHITECTURE

The complete guide to multi-agent systems. 10 chapters covering LangGraph, CrewAI, OpenAI SDK, AutoGen, Google ADK, MCP & A2A protocols. From single-agent limitations to production deployment.